Verifying Local Edits Under Factual and Editorial Constraints

Achintya Raya Polavarapu
Georgia Institute of Technology
Atlanta, Georgia, United States

Ahmedijad Youssef
Contextual Computing Lab
Georgia Institute of Technology
Atlanta, Georgia, United States

ABSTRACT

A local edit can change the meaning of unchanged text, and a factual revision can still violate an editorial rule. We study when a verifier can accept an edited document and what information would justify checking less. A controlled benchmark separates candidate context, factual support, and editorial compliance. Complete context reduces factual errors admitted by the 8B verifier, while the 1.7B joint verifier rejects every constructed error but also 26/32 valid controls. In an initial 100-document SummEdits test, an edit-focused prompt achieves 82% accuracy versus 75% for a same-information baseline: invalid acceptance falls from 22/50 to 14/50, with valid rejection rising from 3/50 to 4/50. A two-stage claim-extraction ablation matches development accuracy at higher cost. A separate test on 50 unused source pairs adds a detailed-checklist control and a second model family. Qwen3-8B reaches 83% versus 80% for the checklist; the three-point difference remains uncertain (95% source-cluster interval [0,7]). Llama-3.2-3B instead reaches 51% versus 61% and rejects 49/50 valid edits. Thus edit-focused prompting, which is not a new algorithm, does not show a consistent advantage across the tested checkpoints. Separately, we characterize the exact minimum worst-case expected rechecking cost under perfect claim queries, assumed nested-scope miss bounds, and distinct valid-retention and false-acceptance constraints. The experiments do not establish those oracle assumptions. The results concern fixed checkpoints and selected benchmarks, not natural editorial validity or downstream benefit.

1 INTRODUCTION

A text-generation system that edits a description must check whether the revision is acceptable. Even a short replacement can change the meaning of the surrounding sentence: "with Tavi's help" may be supported in isolation but false when attached to the wrong actor (Figure 1). A factually true sentence may also violate an editorial rule about what the opening should emphasize. Verification therefore depends on what the checker sees and what it is asked to judge.

A useful verifier must also retain valid candidates. Rejecting every revision prevents all invalid acceptances, but discards every acceptable edit as well. Netflix's judge lifecycle treats failure detection, retention of good explanations, and the quality of rejection rationales as separate concerns [6]. In our offline evaluation, the requested criterion matters too: a "no" can be correct under a joint factual–editorial rule and incorrect under a factual-only rule. Enforcing the editorial rule should not earn credit for factual accuracy.

Our central question is: *When is local verification enough, and what additional information is needed to justify checking less?* The experiments examine assembly context, factual and editorial criteria, and edit-focused instructions. The theory asks what coverage information would justify restricting checks to selected assertions. The paper makes four contributions:

Source evidence
Mira searches for missing navigation charts. Oren watches sealed cargo crates.
Tavi helps Mira search for the charts. Tavi does *not* help Oren watch the crates.

Identical input fragment in both cases: "with Tavi's help."

Supported assembly
Mira searches for missing navigation charts **with Tavi's help.** [Remaining sentences unchanged.]

Contradictory assembly
Oren watches sealed cargo crates **with Tavi's help.** [Remaining sentences unchanged.]

Figure 1: A grammatical phrase has different truth conditions after assembly. The diagram excerpts the affected sentences from two 20–40-word candidates; bracketed comments are explanatory and were never sent to the model. The fragment-only condition sees the same evidence and fragment for both.

1. **A controlled separation of context and criteria.** We evaluate 144 descriptions with distinct factual and sentence-ordering labels through 864 primary judgments. We distinguish what the fragment, whole-factual, and joint checks are asked to judge from what happens when their verdicts gate the complete text. The evaluation holds out lexical families within one grammar; we report a 48-judgment exploratory diagnostic separately.
2. **Evidence that low invalid acceptance can conceal valid-edit rejection.** The 1.7B joint check rejects every constructed error but also 26/32 valid controls. The 8B check behaves differently. Reporting invalid acceptance alone would hide the smaller model's rejection of valid edits.

3. **Held-out factual comparisons with costs and residual errors.** In the initial test on 100 source-disjoint SummEdits documents, Delta-prompt reaches 82% accuracy versus 75% for the same-information Matched baseline. Eight invalid acceptances are corrected and one valid rejection is added. We report its costs, remaining errors, and negative two-call development ablation. A separate comparison on 50 unused source pairs adds a detailed checklist and a second model family. Its uncertain Qwen benefit and severe valid-edit rejection in the tested Llama configuration limit the initial claim (Figure 8).
4. **An exact cost characterization for restricted rechecking.** We characterize the minimum worst-case expected rechecking cost under fallible nested-scope coverage bounds and separate valid-retention and false-acceptance constraints. The result assumes perfect claim queries with additive costs; an optimal policy mixes at most two scopes. Section 8 explains the result, with proofs in Appendix E.

The experiments diagnose failures of context and decision criteria; the theory specifies what a restricted-checking guarantee would require. They are complementary and do not validate one another. The initial external comparison uses one 8B checkpoint; the separate replication uses Qwen3-8B and Llama-3.2-3B-Instruct. Neither comparison measures audience or production benefits. The oracle assumptions are not established for the evaluated prompts, and no priority claim is made for the cost characterization.

2 RELATED WORK

Factual consistency and atomic support. FactCC learns factual-consistency decisions from rule-based transformations of source sentences and uses auxiliary source-evidence and inconsistency spans [7]. FActScore instead decomposes long-form generations into atomic assertions and measures the fraction supported by a designated knowledge source [10]. FactCC checks consistency and localizes errors; FActScore measures support for individual assertions. We instead evaluate a binary decision to accept a complete edited description, with separate labels for factual support and compliance with a stated editorial rule. We do not retrain or compare against either system; the development-only claim-extraction ablation tests an additional decomposition stage within our own prompt pipeline.

Localized edits as an evaluation setting. SummEdits constructs factual-consistency examples by editing verified seed summaries across ten domains [8]. Its GPT-4 oracle experiment also supplies the seed summary, allowing comparison with the edited version. SummExecEdit extends this setting with executable substring replacements and evaluates factual-error detection and error explanations [14]. Localized factual edits and access to original summaries are therefore established evaluation settings. We use SummEdits rather than SummExecEdit and measure binary acceptance and inference cost, without evaluating explanation quality. We ask a narrower question: given the same source, original, edited summary, and differences, does an instruction emphasizing changed relationships perform differently from a generic factual instruction? We retain the benchmark labels on an initial balanced, source-disjoint 100-document test sample and a separate two-model, source-paired replication. Delta-prompt is a prompt variant; this comparison does not introduce a new verification algorithm or reproduce the full benchmark evaluation.

Controlled behavioral diagnosis. CheckList organizes behavioral tests around linguistic capabilities and expected responses, including invariance and directional changes, and supports template-based construction [12]. Chen et al. separately measure verdict invariance under construct-preserving edits and sensitivity to construct-changing edits, showing why stable verdicts alone do not establish construct validity [16]. Our controlled cases apply behavioral diagnosis to attachment, polarity, and sentence ordering. We assign two explicit labels to the same candidates: a true ordering violation fails the editorial target but satisfies factual support. Valid controls reveal over-rejection, while cases with identical fragment inputs show which distinctions that view cannot resolve. These error rates address a related concern but do not estimate their invariance–sensitivity profile. The findings apply to the authored grammar and its lexical variants. The external factual test evaluates different documents under a separate protocol.

Judges as acceptance gates. Kong et al. describe a Netflix judge lifecycle grounded in human rubrics, refinement, bounded revision, and monitoring [6]. Their evaluation already distinguishes rejection of bad explanations, retention of good explanations, and agreement on rejection rationales. We use construction labels to ask whether a verdict is correct under the requested criterion and what happens when it is used to gate joint factual–editorial acceptance. A rejection of factually true text can be correct for that joint gate while still being a factual-classification error. Our measurements cover prompt behavior and inference costs; they do not evaluate a deployed recommendation system or downstream user benefit.

Rubric composition. Yu et al. study changes in a rubric's verdict when other rubrics are co-present and train judges by self-distilling isolated rubric judgments [15]. Their objective concerns consistency across rubric contexts; isolated judgments need not be correct. Our factual-only and joint conditions change the requested acceptance target, so their aggregate verdict differences do not by themselves establish rubric interference. The separate rule-deletion diagnostic retains the factual target while removing the editorial rule from the evidence, but does not identify an internal mechanism. We do not evaluate their training method.

Editing and steering. LaserTagger and the Levenshtein Transformer support generation through edits [5, 9]; steering evaluation documents unintended attribute changes [4]. Our scope is verification of fixed candidates. Preserving most of the wording does not ensure that the assembled text remains factual or complies with an editorial rule.

Verification and costly queries. Randomized certificate complexity studies the cost of verifying a claimed Boolean input; Aaronson gives a nonadaptive characterization using all-agreement query paths [1]. Learning-augmented query policies study costly information acquisition with potentially wrong predictions [2]. Stochastic Boolean function evaluation considers query costs under probabilistic input models, with expected verification and evaluation costs that can differ [3]. Our oracle analysis fixes nested scopes and bounds their probability of missing every false assertion, while imposing separate valid-retention and invalid-acceptance constraints. It uses established coupling and convexity arguments and makes no priority claim for this formulation. Its additive claim-query model is separate from the empirical LLM calls.

3 TASK AND CONSTRUCTED BENCHMARK

3.1 Two Complete-Description Targets

A case contains source evidence E, canonical description x, permitted character span $[s, e)$, and replacement r. Assembly is deterministic:

$$y = x_{<s}\, r\, x_{\geq e}. \tag{1}$$

The factual label $f(E, y) \in \{0, 1\}$ records whether the constructed event, participant, and polarity claims agree with the source world. The policy label $p(E, y) \in \{0, 1\}$ records compliance with this explicit rule: *the first sentence must concern the central work activity; a true minor-theme event may appear only after it.* Joint acceptance is $j(E, y) = f(E, y)p(E, y)$.

The ordering label measures compliance with the stated sentence-position rule. A true minor event used as the first sentence satisfies f and fails p, providing a controlled separation between factual support and editorial compliance.

3.2 Construction, Certificates, and Split

Six lexical scenario families describe a harbor, archive, orchard, observatory, ferry, and theater. Each family has four entity variants and each variant has six candidate descriptions. There are 24 base scenarios and 144 candidates. The harbor and archive families form development: eight scenarios and 48 candidates. The other four families form evaluation: 16 scenarios and 96 candidates. Families, case order, and construction labels were fixed before model collection. All families share the same sentence grammar; evaluation tests new names, activities, objects, and settings within that grammar, not generalization to unseen syntax.

Each source world explicitly states two actors' activities, whether a helper assists each activity, two true minor events, the central activity, and the ordering rule. The source includes a negative helper relation so that the wrong-actor case directly contradicts the evidence. The canonical and all assembled candidates contain 20–40 words. Primary replacements cover whole prepositional phrases or sentences. Metadata for a noun-bisecting stress construction is stored separately and excluded from every primary model call.

Labels are derived from clause certificates specifying factual support and the event class of each sentence. The checker operates on a closed dictionary of authored constructions. Programmatic validation checks source records, exact assembly, class balance, span boundaries, gold-label sidecars, and hashes. The author constructed the reference labels with AI assistance. Section 10 describes their scope and validation.

3.3 Why the Replacement View Loses Information

The accepted-helper and wrong-actor cases share exactly the same source evidence and replacement fragment. Only their assembly location differs. Because that location and the surrounding sentence are omitted from F, the two model inputs are byte-identical before rendering and have identical rendered token IDs. F cannot determine which complete candidate would receive its verdict. A second collision pairs a true minor-event replacement used after the opening with the same sentence used as the opening; these share factual truth but have different joint labels.

Figure 1 illustrates the first collision using a development scenario. The source explicitly states both helper relations. These paired cases show what is lost when assembly context is withheld. They do not imply that the verifier's answer is wrong for the fragment task it was given.

4 CONTROLLED EXPERIMENT: PROTOCOL

4.1 Information and Criterion Ablations

We run three fixed prompts on every candidate (Figure 2). **F: fragment fact** asks whether the factual content expressed in r is supported, without inferring its unseen subject or surrounding event. **W: whole fact** asks whether all claims in y are supported, including relations created by modifier attachment, and explicitly excludes ordering and prominence. **J: whole joint** asks whether y is supported and obeys the ordering rule. The evidence is identical across all three conditions, including the rule. No condition receives a gold label, case identifier, family, profile, construction type, or edit-location metadata.

F deliberately omits the assembly context. Its acceptance on a wrong-actor assembly may be correct for the fragment it was shown. We therefore do not score F's declared fragment-task accuracy using the full-candidate factual label. Its measured full-candidate errors describe the consequences of *using fragment acceptance as an assembly gate.* Likewise, W is supposed to accept a true ordering violation; such acceptance is not a factual error. Joint gate-use metrics and declared-target metrics are reported separately.

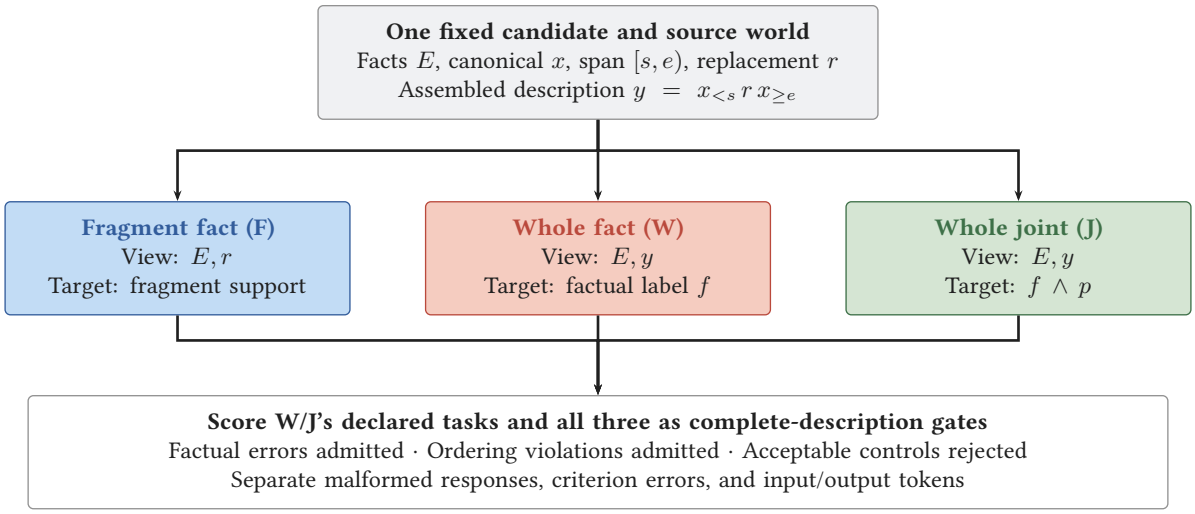


Figure 2: The three conditions vary the visible candidate or the requested criterion. Every condition receives the same source evidence, including the declared ordering rule. W is explicitly told to ignore that rule; J must apply it. Case identifiers, construction labels, profiles, and the original edit location are absent from the prompts. F is an information ablation, not a recommended production verifier.

Construction	Controlled change	f	p
Acceptable helper	Attach the helper to the supported actor/activity	1	1
Acceptable minor event	Put a true minor event after the central opening	1	1
Wrong actor	Attach the same helper phrase to the unsupported actor	0	1
Reversed polarity	Negate the source's affirmative central activity	0	1
Minor event first (A)	Replace the opening with true minor event A	1	0
Minor event first (B)	Replace the opening with true minor event B	1	0

Table 1: Six constructions per scenario. Each occurs eight times in development and 16 times in evaluation. A value of 1 denotes a true construction label; all six satisfy the mechanical assembly and length contract. Frequencies are fixed by design.

4.2 Frozen Local Execution

We evaluated frozen four-bit Qwen3-8B as the primary verifier and Qwen3-1.7B as a smaller-system replication [11, 13]. Each model produced 432 judgments on the same 144 cases under all three conditions. Model-specific template handling produced identical effective input-token sequences for corresponding requests. Generation used greedy decoding, temperature zero, a 128-token output limit, batches of four, and no retries. All runs executed locally on an Apple M3 with 16 GiB memory, with $0 in external API spending. The artifact retains input and output token IDs, raw responses, prompts, model revisions, weight hashes, runtime versions, and pre-call records.

For each model, an initial response-contract check collected 18 judgments on the six cases of the first development scenario (the `smoke` stage). These belong to the 144 development judgments. The other 126 development and 288 evaluation judgments were then collected. The three prompts were frozen before the smoke, and the observed factual-only rejection of ordering violations did not trigger prompt changes. No evaluation outputs were used for prompt selection.

The primary response must be one JSON object containing exactly `verdict: "yes"` or `verdict: "no"`. Duplicate keys, extra fields, non-JSON wrappers, and other values are failures. A separately specified wrapper normalization is diagnostic only; it never replaces the primary verdict. A malformed response is an operational rejection and an incorrect classification, rather than a correct semantic "no."

Appendix B records the exact prompts, model identities, runtime configuration, and reproducibility checks.

4.3 Endpoints, Controls, and Uncertainty

For each model and condition, we report the fraction of factual-error cases accepted (32 evaluation cases), ordering-only violations accepted (32), and acceptable controls rejected (32). Acceptances require a valid "yes." Rejection counts distinguish explicit "no" from malformed output. We also report malformed counts, input tokens, and returned completion tokens. Declared-target accuracy uses f for W and j for J, and is undefined here for F because no separate fragment-gold certificate is supplied.

Always accept and always reject are deterministic reference points. The former also describes the accept/reject behavior of a gate that checks only the construction's mechanical contract, which all candidates meet. They require no model invocation. Always reject has zero false acceptance but rejects every acceptable control. We additionally report full-candidate joint error, with malformed outputs counted as errors, and its equal-class-weight version. These coincide here because the three construction classes have equal size; those weights are not population frequencies.

Three paired contrasts were specified before evaluation: W

minus F on factual-error admission; J minus W on ordering-violation admission; and J minus W on acceptable-control rejection. Primary descriptive 95% percentile intervals use 5,000 paired bootstrap resamples of the four held-out lexical families, seed 20261008. Resampling keeps every scenario, construction, and condition within a family together. A secondary 16-scenario bootstrap is conditional on these families. Four families provide little distributional coverage: degenerate intervals or zero observed errors are not population guarantees or evidence of equivalence. No hypothesis-test winner claim is made.

4.4 An Exact-View Information Bound

For a fixed evaluation target $t \in \{f, j\}$, partition cases into groups G with identical visible messages. Verify that each such group also has identical rendered input tokens. Let $n_1(G)$ and $n_0(G)$ count the two target labels. Any deterministic binary classifier using only that view must have empirical error at least

$$L_t = \frac{1}{N} \sum_{G} \min\{n_1(G), n_0(G)\}. \tag{2}$$

Each group receives one decision; choosing its majority label minimizes its errors. The bound also holds in expectation for a randomized rule whose output distribution depends only on that view. Counting abstentions as errors cannot improve it. This elementary calculation applies to the finite set; it is neither a new theorem nor an estimate of population Bayes risk. A zero bound for a richer view says only that its exact inputs do not contradict their labels; it does not imply that a model can classify them correctly.

5 CONTROLLED EXPERIMENT: RESULTS

5.1 Primary Evaluation

The two models produced 864 judgments: 288 development and 576 evaluation judgments over 48 development and 96 evaluation candidates. Models and conditions reuse the same candidates; judgments are not independent scenario samples. All 864 responses satisfy the strict JSON contract, with zero retries or truncated outputs. Primary and normalized-format diagnostics agree. Table 2 reports every evaluation candidate; no row is discarded for a failed model response.

For 8B, F admits 16/32 factual errors, W admits 6/32, and J admits 2/32. All 16 reversed-polarity cases are rejected in every condition. The residual factual accepts are wrong-actor attachments: F accepts 16/16, W 6/16, and J 2/16. Both remaining J errors are in the orchard family. For example, case `b11_c03` says that Yuri counts crates "with Zola's help," despite explicit evidence that Zola does not help Yuri count them. Complete context makes the distinction available, but does not ensure a correct verdict.

J accepts no ordering-rule violations (0/32), compared with 17/32 under W and 32/32 under F. Every 8B condition accepts all 32 joint-acceptable controls. Thus J's observed reduction in invalid accepts does not result from rejecting everything. Its joint classification error is 2/96, compared with 23/96 for W and 48/96 for F. Figure 3 separates these outcomes by error class. These are complete-gate diagnostics across different requested targets, not three interchangeable factual-classification accuracies.

The predeclared 8B W-minus-F contrast reduces factual-error admission by 10/32 cases, or 31.25 percentage points. The J-minus-W contrast reduces ordering-violation admission by 17/32, or 53.125 points. Acceptable-control rejection does not change on these cases. Table 3 retains all three contrasts and the four-family descriptive intervals. Observing zero acceptable-control rejection in this collection does not establish equivalence or a low population risk.

Concrete successes and failures. In evaluation case `b13_c03`, the candidate states that Evan protects lockers "with Faye's help," although source record E2 explicitly says Faye does not help Evan. The 8B fragment check accepts; its W and J checks correctly reject. In `b11_c03`, the parallel attachment to Yuri is accepted by all three 8B conditions despite the explicit negative relation. In one case, providing the complete text allows a correct rejection; in the other, the error persists even with that context.

Case `b09_c01` is a valid edit: the source says Tala helps Rina repair pumps, and the edited opening states that activity with Tala's help. The 8B joint verifier accepts it; the 1.7B joint verifier rejects it. In `b09_c06`, a true minor event replaces the opening. The 8B W and J verdicts are both "no": one is wrong under the factual target, and the other is correct under the joint target. Figure 4 shows invalid acceptance and valid-control rejection together; their interpretation depends on the declared criterion.

5.2 Factual-Target Errors Hidden by a Joint Gate

W also rejects 15 factually true ordering violations: 1/16 of minor variant A and 14/16 of variant B. These are errors against its explicit factual-only target. Together with its six factual false accepts, they produce 21/96 declared-target errors, or 78.1% accuracy. J's declared joint-target accuracy is 94/96 (97.9%). These accuracies use different gold targets and cannot be compared as scores on the same task.

W does not reject the two ordering variants equally, so the results do not show uniform enforcement of the ordering rule. The verdicts establish factual-task errors but contain no reasoning trace. Source wording, content coverage, event identity, and criterion attention are possible contributors. Although W rejected both ordering variants in the smoke check, it did not do so for every held-out case.

5.3 Smaller-Model Replication

The 1.7B joint check rejects more invalid candidates than its larger counterpart, but also more valid ones. F admits 15/32 factual errors, W 12/32, and J 0/32. J also rejects all 32 ordering violations. However, it rejects 26/32 acceptable controls (81.25%), compared with 3/32 for W and 1/32 for

Model	Condition	Factual accepted	Order accepted	Acceptable rejected	Malformed /96	Target accuracy
8B	F: fragment fact	16/32	32/32	0/32	0	–
8B	W: whole fact	6/32	17/32	0/32	0	78.1%
8B	J: whole joint	2/32	0/32	0/32	0	97.9%
1.7B	F: fragment fact	15/32	32/32	1/32	0	–
1.7B	W: whole fact	12/32	25/32	3/32	0	77.1%
1.7B	J: whole joint	0/32	0/32	26/32	0	72.9%
Analytic	Always accept	32/32	32/32	0/32	0	–
Analytic	Always reject	0/32	0/32	32/32	0	–

Table 2: Held-out evaluation: 96 candidates per condition and model. Each error-class denominator is 32. F/W/J are defined in Figure 2. *Order accepted* is a joint gate-use diagnostic, not a factual error for F or W. Declared-target accuracy scores W against factual support and J against factual support plus the rule; F has no separately certified fragment gold.

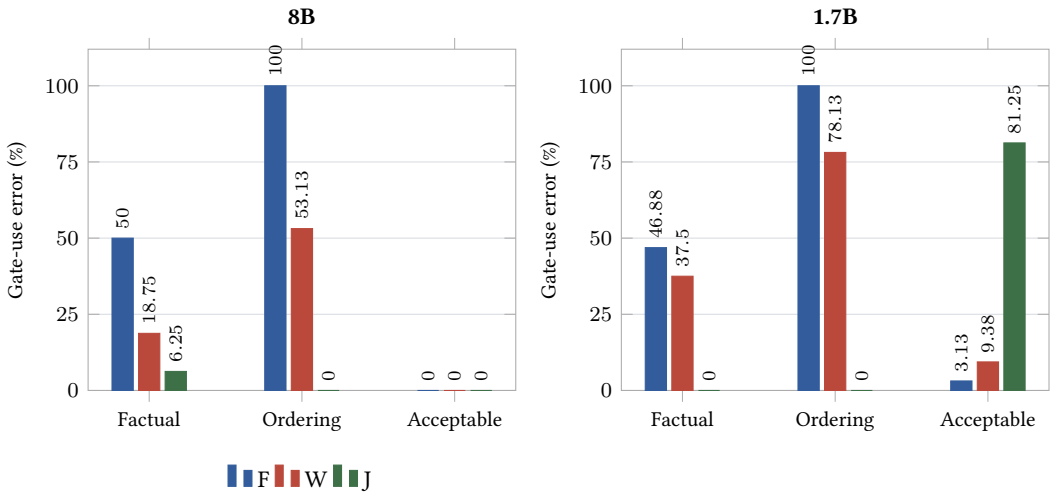


Figure 3: Gate-use outcomes on the same held-out candidates. Factual and ordering bars show erroneous candidates accepted; acceptable bars show controls rejected. Lower is better for the complete joint gate, but the requested tasks differ. In particular, rejecting a true ordering violation is an error under the stated factual-only target of W. Counts and malformed rates appear in Table 2.

Model	Contrast	Outcome	Difference	95% interval
8B	W minus F	Factual errors admitted	-31.2	[-46.9,-9.4]
8B	J minus W	Ordering violations admitted	-53.1	[-62.5,-43.8]
8B	J minus W	Acceptable controls rejected	0.0	[0.0,0.0]
1.7B	W minus F	Factual errors admitted	-9.4	[-28.1,0.0]
1.7B	J minus W	Ordering violations admitted	-78.1	[-100.0,-56.2]
1.7B	J minus W	Acceptable controls rejected	71.9	[62.5,90.6]

Table 3: Predeclared paired evaluation contrasts, in percentage points, with the subtraction order named in each contrast. Every row has 32 eligible cases and four family clusters. Intervals use 5,000 paired family resamples and are descriptive; repeated or degenerate endpoints do not imply population certainty.

F. The predeclared J-minus-W acceptable-rejection contrast is therefore $+23/32$, or $+71.875$ percentage points, with a descriptive family-bootstrap interval of [62.5,90.625] points. The stricter check therefore comes at a substantial cost to valid edits on this benchmark.

The joint gate error falls from 40/96 under W to 26/96 under J, but that aggregate improvement hides the loss of 23 acceptable controls. J accepts only six candidates in total. The always-reject control reaches zero false acceptance simply by discarding every acceptable edit as well. Declared-target errors are 22/96 for W and 26/96 for J. This comparison uses two model configurations with matched inputs; it does not establish model size as the cause of the difference.

5.4 Exact-View Limits and Token Accounting

Across the 96 evaluation cases, F has 64 unique visible inputs. Sixteen groups contain opposite factual labels and 32 contain opposite joint labels. Equation 2 therefore gives error floors of 16/96 (16.7%) for assembled factual validity and 32/96 (33.3%) for joint validity. W and J have 96 unique inputs and no contradictory exact-view groups. Neither model

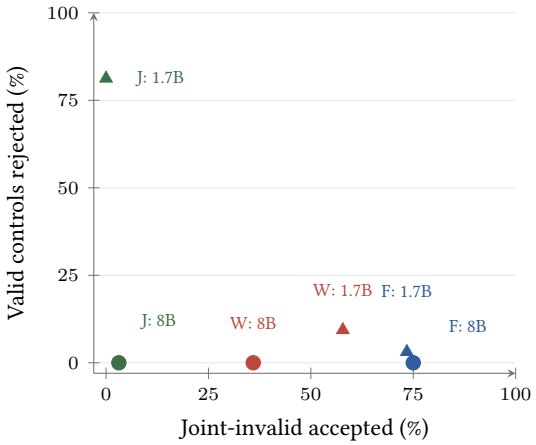


Figure 4: Controlled evaluation as a joint acceptance gate: factual or ordering-invalid candidates accepted (64 cases) versus acceptable controls rejected (32 cases). Lower values are better for this gate. Circles denote 8B and triangles 1.7B. F, W, and J have different declared targets; these points are not a same-task accuracy comparison. Shared construction families preclude treating the points as independent Bernoulli trials.

produces different verdicts within an identical-view group in this collection. W and J receive distinct inputs for every candidate, but their remaining errors show that access to this information does not ensure a correct verdict.

Each response contains six returned tokens. The full primary collection uses 413,080 input and 5,184 output tokens. The exploratory diagnostic adds 19,593 input and 288 output tokens: total controlled-study inference is 912 judgments, 432,673 input tokens, and 5,472 output tokens. Primary and exploratory judgments have separate ledgers, and all reported token totals include every call in their respective collections. The three prompts differ in instruction length as well as candidate view. Their token differences therefore describe these exact requests, not a general cost advantage of fragment or whole-description verification.

5.5 Exploratory Rule-Deletion Diagnostic

After inspecting the 8B smoke, we separately froze a development-only diagnostic: use the same W instruction and candidates while deleting E6, the explicit ordering rule. E0–E5 remain byte-for-byte unchanged. This produced 48 additional 8B judgments; the primary evaluation was neither altered nor rerun.

The original W development run has 12/48 factual-task errors, all false rejections of true ordering violations. Without E6, all 48 development judgments match the factual construction labels. Exactly 12 responses change from "no" to "yes"; the other 36 stay the same. Factual-error admission remains 0/16 and acceptable-control rejection remains 0/16. Figure 5 presents these results separately from the held-out comparison.

Removing the rule changes verdicts on two development families, consistent with interference from irrelevant policy information. This observation alone does not identify the cause. Deletion also changes sequence length and token positions; the diagnostic's batch companions differ from the original mixed-condition batches. It neither identifies internal reasoning nor proves that policy content alone caused the errors. Appendix C records its separate protocol and full accounting.

Check	Input total	Input mean	8B output	1.7B output
F	44,051	458.9	576	576
W	44,747	466.1	576	576
J	48,779	508.1	576	576

Table 4: Evaluation token accounting per model, across 96 cases for each condition. Effective input IDs are matched between models. Returned tokens exclude stop tokens and are not a monetary or latency estimate.

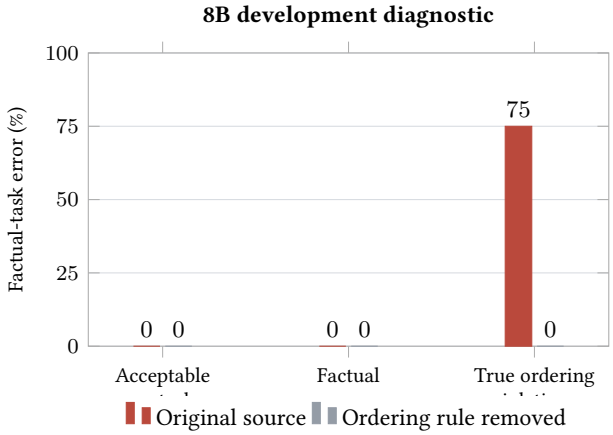


Figure 5: Post-smoke exploratory sensitivity on 48 development cases, with 16 cases per class. After removal of E6, 12 false rejections of true ordering violations become factual-task accepts; the other 36 verdicts remain unchanged. No evaluation cases were rerun. Input length, token positions, and batch companions also differ, limiting causal interpretation.

6 INITIAL EXTERNAL VALIDATION ON SUMMEDITS

6.1 Data, Labels, and Frozen Split

SummEdits supplies heterogeneous source documents, seed summaries, benchmark-generated edited summaries, and binary factual-consistency labels [8]. It combines existing corpora with ChatGPT-generated sales-call and sales-email sources; these synthetic source domains contribute 18/100 test cases. The prepared artifact contains 6,348 normalized rows and 216 distinct exact-source groups. We group by SHA-256 of the complete normalized source document, deterministically allocate 30% of source groups to development and the remainder to test, and sample at most one changed summary per group. The fixed seed is 20261008. No document-length cap was applied. Within these partitions, we select prespecified samples of 40 development cases (20 valid, 20 invalid) and 100 test cases (50 valid, 50 invalid), with no exact source shared between the splits. This balanced,

source-filtered sample has a different distribution from the full benchmark.

In the test sample, source documents contain a median of 389.5 whitespace-delimited words (range 39–2,090), and edited summaries a median of 42 (22–86). The median lexical change is four words (1–17), counting the larger of removed and inserted word counts for each nonmatching diff block and summing across blocks. The median is three edit blocks; only 24/100 examples have one contiguous block. The external study therefore includes small lexical edits at several locations, whereas the controlled benchmark replaces a single span. All ten benchmark domains are represented.

The published factual-consistency labels are retained exactly. They are not labels of member preference, first-sentence ordering, or editorial quality. No new human judgments were collected. Source grouping prevents exact-document overlap across these splits; it does not rule out related documents or prior exposure during model training. The local execution records retain the dataset revision, case IDs, data hashes, and original research-kit selection code. The public package restores this initial sample from saved identifiers; it does not distribute that original sampler. The separate source-paired replication includes its selection implementation.

6.2 Three Fixed Factual Verifiers

All three initial methods use the same frozen Qwen3-8B checkpoint and require factual support for the *complete edited summary*. The common system message excludes style and attractiveness. **Whole** receives the source and edited summary. **Matched** also receives the original summary and word-level differences, but treats these as context rather than evidence of correctness. **Delta-prompt** receives exactly the same data fields as Matched and explicitly directs attention to changed relationships, negation, attribution, temporal order, and interactions with unchanged text, while still checking all remaining factual claims.

Matched is the same-information comparison for Delta-prompt: only the decision instruction differs, although its length is not token matched. Whole additionally differs in available information. Word differences are computed by `difflib.SequenceMatcher` with `autojunk=False`; at most 12 change chunks are shown, followed by a count of further chunks. Complete original and edited summaries remain visible to both edit-aware conditions. The three conditions are prompt variants, not trained algorithms, and are distinct from controlled F/W/J.

The first five development cases were retained when collection expanded to all 40; no completed responses were regenerated. Baseline prompts remained fixed through development. After a separate claim-extraction ablation, all three baseline methods and all 300 test requests were frozen before the final run. The test set was executed once, in saved case order with Whole, Matched, and Delta-prompt for each case. No prompt, label, threshold, algorithm, or method choice was changed using test results.

Generation used greedy decoding, temperature zero, the verified nonthinking chat template, a 64-token output cap, and one request at a time. Every response, rendered prompt, input/output token ID, termination reason, and generation time was cached. This differs from the controlled experiment's 128-token cap and batching; costs are reported within each protocol. The final test wrapper adds pre-call receipts and persistence without changing the original runner's model inputs or decoding.

6.3 Endpoints and Uncertainty

All accuracies in this initial comparison use the same supplied binary label. An invalid acceptance is a valid `yes` on one of the 50 inconsistent test summaries; a valid rejection is a `no` or malformed response on one of the 50 consistent summaries. Strict accuracy gives no credit for malformed responses, even when an operational rejection blocks an invalid edit. Always accept and always reject each obtain 50% accuracy on this balanced test set, but have opposite class-specific errors.

We report Wilson 95% intervals for each method's accuracy and two error rates, and paired disagreements on the same cases. Paired differences use 10,000 source-level bootstrap resamples with shared indices across methods, seed 20261008, and floor-index percentile endpoints. Accuracy resamples all 100 sources without class stratification; each error rate resamples its 50 eligible sources.

The main same-information contrast is Delta-prompt minus Matched; the other two method pairs are also reported. Requests, methods, and broad endpoints were frozen before inference; the detailed analysis specification was fixed during collection, before inspection of held-out outcomes. This was a local protocol freeze, not a public preregistration. These intervals describe this selected sample, not the prevalence of errors in deployed descriptions. We make no confirmatory significance or equivalence claim.

Input and completion tokens are summed over every call; stop tokens are recorded separately. Timings cover streamed generation, including prompt processing, but exclude model loading and analysis. Fixed method order and different development sessions prevent interpreting them as randomized latency benchmarks.

6.4 Held-Out Factual Verification

All 300 test calls completed once, with one saved response for each case–condition pair, no retries, no malformed outputs, and no output truncation. Independent replay verifies every rendered prompt, token count, source label, and cached verdict against the frozen requests. The original runner, prepared data, model files, and prior experiment results remain unchanged.

Delta-prompt achieves 82/100 correct, compared with 75/100 for Matched and 74/100 for Whole (Table 5). Invalid acceptance falls from 22/50 under Matched to 14/50 under

Method	Accuracy [95% CI]	IA	VR	Input tokens	Output tokens	Time (s)
Whole	74.0 [64.6,81.6]	23/50	3/50	107,162	600	1,066.59
Matched	75.0 [65.7,82.5]	22/50	3/50	121,675	600	1,194.95
Delta-prompt	82.0 [73.3,88.3]	14/50	4/50	124,375	600	1,217.82

Table 5: Final SummEdits test: 100 sources, with 50 valid and 50 invalid summaries. All three methods use the same factual target. Accuracy and its Wilson interval are percentages; IA is invalid accepted, and VR is valid rejected. Tokens and generation time sum all 100 calls per method. Returned output excludes 100 EOS tokens per method. Times are observed sequential generation costs, not randomized latency comparisons.

Delta-prompt, while valid rejection increases from 3/50 to 4/50. Figure 6 shows the three endpoints with Wilson intervals. Delta-prompt therefore increases observed accuracy and reduces invalid acceptance, at the cost of one additional valid rejection; it does not eliminate either type of error.

For the same-information comparison, the paired accuracy difference is $+7$ percentage points with a descriptive 95% interval of $[2, 13]$ (Table 6). Delta-prompt corrects eight Matched invalid acceptances and introduces one rejection of a benchmark-valid summary. It introduces no new invalid acceptance relative to Matched on these cases. The three methods disagree on 14 distinct examples; pairwise disagreement counts are seven for Matched versus Whole, twelve for Delta-prompt versus Whole, and nine for Delta-prompt versus Matched. The saved per-example table records every case, original label, verdict, and disagreement, including common errors.

Delta-prompt has higher accuracy than both baselines on development and test. The ten-point development difference from Matched becomes seven points on test. However, the development observation of zero valid rejections does not persist: all three test conditions reject valid summaries. Whole and Matched also reverse their near-tied development ordering. We therefore report the 40-case development results separately from the held-out comparison.

The final test consumes 353,212 input and 1,800 returned output tokens, plus 300 recorded EOS tokens, over 3,479.36 seconds of generation. Delta-prompt uses 124,975 input-plus-output tokens versus 122,275 for Matched, a 2.21% increase. Their payloads and completion lengths match; the difference is the 27-token instruction increment on each of 100 requests. Observed generation times are 1,217.82 and 1,194.95 seconds, respectively. Figure 7 plots cost against accuracy separately for development and test.

Illustrative test cases. In `sales_email_9_14`, the source offers a "10% discount on your first year of subscription," while the edited summary gives a "discounted rate of 9%." Whole and Matched accept this benchmark-invalid summary; Delta-prompt correctly rejects it. In `Bki1Ct1AW_23`, "non-negative matrix factorization" becomes "negative matrix factorization." All three methods accept this invalid edit despite the change in meaning.

Conversely, `billsum_train_18187_6` is benchmark-valid: "enter into agreements" becomes "join forces," and "permit" becomes "give … the power." Whole and Matched accept, while Delta-prompt rejects. This is its sole accuracy regression relative to Matched. These post-hoc examples illustrate recorded outcomes and retain the benchmark labels. The verdict-only responses do not reveal which phrase caused a decision or its internal reasoning.

6.5 Negative Development Ablation: Extract Then Verify

The additional condition `delta_claims_v1` first compares the complete summaries and extracts factual assertions whose meaning changed. It includes entities, relationships, attribution, negation, quantities, and temporal or conditional scope, with surrounding unchanged context. A separate call receives the source, both summaries, and the extracted claims, checks the claims and the entire edited summary, and returns a verdict plus the failed assertions. Extraction is a guide, not evidence; the verifier must also check claims the extractor omitted. Both prompts are generic and contain no observed-error examples or case-specific rules.

This condition was developed after examining baseline development errors, frozen before its own calls, and run only on the existing 40 development cases. It uses the same model and greedy nonthinking decoding, with extraction and verification budgets of 768 and 512 tokens. Both stages are charged to its cost. Malformed extraction or verification would count as a strict pipeline error and operational rejection, with any recognizable model verdict retained separately. All 80 calls completed without parsing errors or output truncation.

The two-call condition ties Delta-prompt at 31/40 correct (77.5%), with nine invalid accepts and zero valid rejects (Table 7). It fixes `billsum_train_8220_19` and regresses on `rklVOnNtwH_28`; all other verdicts agree. Total tokens increase from 46,494 to 74,995 (1.61 times), and recorded generation time increases from 375.30 to 1,026.39 seconds (2.73 times). The extra inference does not improve aggregate accuracy on these development cases. This negative result applies to the tested ablation; it does not rule out benefits from other uses of explicit claims. It was not run on the final test set.

A separate AI-assisted review flagged seven development cases for possible label ambiguity or inherited source–summary discrepancies. The flags include source quotations, require human confirmation, and change neither labels nor scores. No adjusted-label result is reported.

Contrast (A minus B)	Accuracy	Invalid acceptance	Valid rejection	Only-correct A:B
Matched − Whole	$+1$ $[-4, 6]$	-2 $[-12, 8]$	0 $[0, 0]$	4:3
Delta-prompt − Whole	$+8$ $[2, 15]$	-18 $[-30, -6]$	$+2$ $[0, 6]$	10:2
Delta-prompt − Matched	$+7$ $[2, 13]$	-16 $[-26, -6]$	$+2$ $[0, 6]$	8:1

Table 6: All three paired test comparisons. Differences and descriptive paired-bootstrap 95% intervals are in percentage points; accuracy uses 100 sources, and each error rate its 50 eligible sources. The last column counts cases where only A is correct versus only B. The [0,0] interval reflects identical observed valid-case outcomes, not established population equivalence. No multiplicity-adjusted significance claim is made.

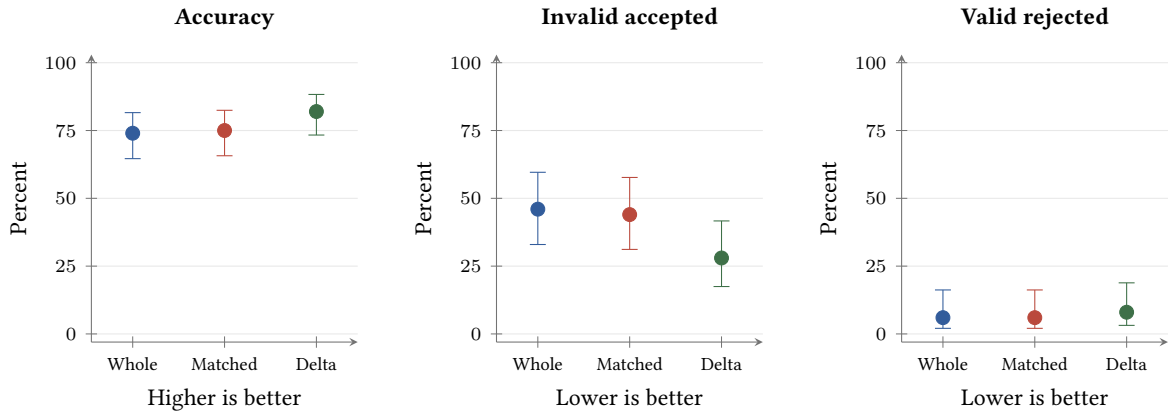


Figure 6: Final SummEdits test performance on 100 cases (50 valid, 50 invalid), with unchanged dataset labels. Delta denotes `delta_prompt`. Error bars are descriptive 95% Wilson intervals for accuracy ($n = 100$), invalid acceptance ($n = 50$), and valid rejection ($n = 50$). Intervals describe marginal proportions, not paired differences or domain-population coverage.

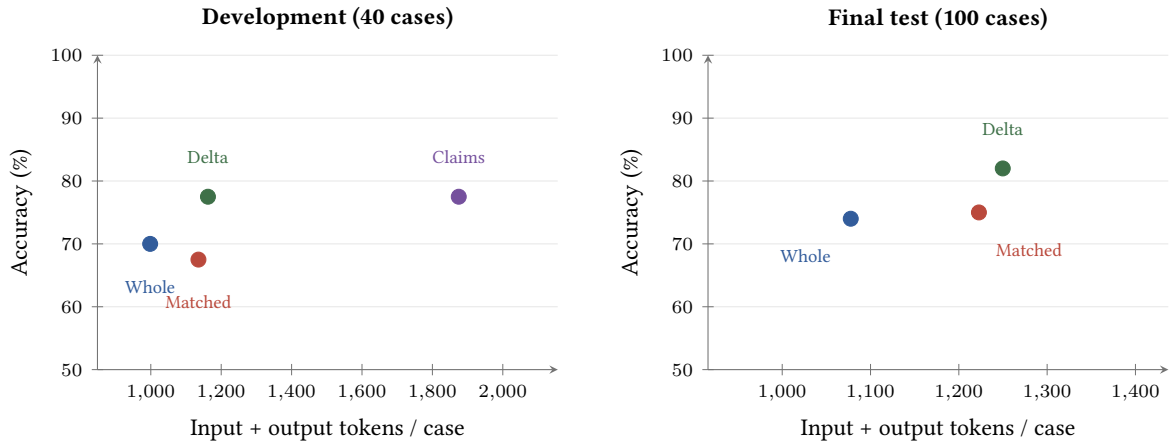


Figure 7: Observed tokens per case versus verification accuracy, shown separately for development and final test. Tokens sum inputs and outputs over all calls, excluding EOS. Claims denotes the two-call `delta_claims_v1` development ablation; it matches Delta accuracy while using more tokens. The panels contain different source documents and have independent cost axes, so their positions do not support a joint development–test ranking.

Development	Correct	IA	VR	Tokens
Whole	28/40	12/20	0/20	39,927
Matched	27/40	13/20	0/20	45,414
Delta-prompt	31/40	9/20	0/20	46,494
Delta-claims	31/40	9/20	0/20	74,995

Table 7: External development results, separate from the final test. IA: invalid accepted; VR: valid rejected. Tokens sum input and completion tokens, excluding EOS. Delta-claims denotes `delta_claims_v1` and charges both calls.

7 INSTRUCTION SPECIFICITY AND SECOND-MODEL REPLICATION

The initial comparison changes the detail of the instruction as well as its emphasis on edits. A verifier might benefit from being told to check negation and attribution even without being directed toward changed text. We therefore add a detailed factual checklist and repeat the comparison in a second model family. This is a separate test on unused sources; it does not replace or enlarge the initial test retrospectively.

7.1 Source Pairs and Frozen Conditions

We exclude all 140 sources used in the earlier development and test sets. Source identity here applies Unicode NFKC normalization and collapses whitespace, retaining case and punctuation. Of the 76 remaining source groups, 71 contain both a changed benchmark-valid edit and a changed benchmark-invalid edit of the same original summary. A deterministic, domain-stratified selection reserves ten source pairs for development and fifty different pairs for test. Thus the test has 100 examples but only 50 source clusters; development has 20 examples from ten sources. Both candidates in a pair share the exact source and seed summary. Original labels remain unchanged, all ten domains are represented, and no length cap is applied. The selection code and exclusion ledger accompany the data hashes.

Matched and **Delta-prompt** retain their initial instructions verbatim. **Checklist** appends a generic factual checklist to Matched: entities, relationships, negation, attribution, quantities, temporal order, consequences, and cross-sentence relationships. It requires checking every factual claim and retaining supported paraphrases. It does not direct special attention to changed relationships. All three conditions receive the same system message and the same source, original summary, edited summary, and token differences. The checklist is a specificity control; it is not an exact token-length control or a factorial isolation of edit focus.

The models are the original four-bit Qwen3-8B snapshot and a four-bit MLX conversion of Llama-3.2-3B-Instruct.[1] Each uses its native chat template, with Qwen thinking disabled and Llama's template date fixed. Decoding is greedy with a 64-token cap, no truncation, one call per condition and example, and no retries. Method order within a case is deterministically shuffled and shared between models; model blocks execute sequentially. The protocol renders and hashes all 720 requests before the first call. Development checks execution and formatting only: prompts, settings, and analysis are already frozen and are not selected using its outcomes. Both development blocks and their analysis finish before test generation begins.

7.2 Paired Analysis

The primary contrast is Delta-prompt minus Checklist accuracy, separately for each model. We also report Delta-prompt minus Matched and Checklist minus Matched, invalid acceptance, valid rejection, malformed replies, tokens, and generation time. Malformed replies receive no accuracy credit and count as operational rejections, as in the initial experiment. Always accept and always reject again obtain 50% accuracy on the balanced pairs.

We use 10,000 paired bootstrap draws of whole source groups, preserving both edits and every method's outcomes; the same draws are used for both models. Percentile endpoints use floor indices and seed 20261009. A separate paired randomization test swaps method labels jointly for both edits of a source, using 100,000 sign-flip draws and a two-sided plus-one p-value. Holm correction applies to the two model-specific primary accuracy tests; secondary comparisons are exploratory. The local freeze precedes all inference but is not a public preregistration. The intervals describe the selected source sample; they do not treat paired edits as independent or establish deployment error rates.

7.3 Results and Inference Cost

Table 8 reports every test condition, and Figure 8 separates accuracy from the two error rates. Table 9 gives the prespecified primary contrasts. The development results appear in Table 13; no method was modified or selected from them.

For Qwen3-8B, Matched, Checklist, and Delta-prompt obtain 81.0%, 80.0%, 83.0% accuracy. Relative to Checklist, Delta-prompt changes accuracy by +3.0 percentage points with source-cluster interval $[0.0, 7.0]$. It corrects 3 benchmark-label errors and introduces 0; invalid acceptance is 15/50 versus 18/50, and valid rejection is 2/50 versus 2/50. Only three of the 50 source clusters contribute a nonzero accuracy difference. The interval endpoint at zero does not rule out worse performance on other samples.

For Llama-3.2-3B, Matched, Checklist, and Delta-prompt obtain 54.0%, 61.0%, 51.0% accuracy. Relative to Checklist, Delta-prompt changes accuracy by -10.0 percentage points with source-cluster interval $[-17.0, -3.0]$. It corrects 10 benchmark-label errors and introduces 20; invalid acceptance is 0/50 versus 10/50, and valid rejection is 49/50 versus 29/50.

The full per-example record lists all three paired method comparisons for each model, including errors shared by the methods. A verdict disagreement identifies different decisions, not the reason for either decision. No human error categories or adjudicated labels are inferred from these model replies. Llama is already rejection-heavy under Matched (39/50 valid edits rejected) and Checklist (29/50); Delta-prompt aggravates that behavior without identifying its cause.

Qwen3-8B uses 135,417 tokens for Delta-prompt and 136,717 for Checklist over 100 calls each; their generation times are 1,405.86 and 1,428.26 seconds. Llama-3.2-3B uses 135,022 tokens for Delta-prompt and 136,322 for Checklist over 100 calls each; their generation times are 605.00 and 613.71 seconds. The 600 test calls contain 0 malformed replies and 0 length-terminated outputs. The checklist adds 13 input tokens per example relative to Delta-prompt in each tokenizer. Token costs are tokenizer-specific, and sequential times do not isolate a prompt or model effect on serving latency.

[1] The upstream Llama-3.2-3B-Instruct model card and the MLX conversion document the second checkpoint. Exact revisions and file hashes are recorded with the experiment.

Model	Method	Accuracy [95% CI]	IA	VR	Tokens	Time (s)	Malformed
Qwen3-8B	Matched	81.0 [74.0,88.0]	18/50	1/50	132,717	1,362.78	0
	Checklist	80.0 [73.0,87.0]	18/50	2/50	136,717	1,428.26	0
	Delta-prompt	83.0 [76.0,89.0]	15/50	2/50	135,417	1,405.86	0
Llama-3.2-3B	Matched	54.0 [47.0,61.0]	7/50	39/50	132,322	598.14	0
	Checklist	61.0 [54.0,68.0]	10/50	29/50	136,322	613.71	0
	Delta-prompt	51.0 [50.0,53.0]	0/50	49/50	135,022	605.00	0

Table 8: New paired-edit test: 100 examples from 50 previously unused sources, with one valid and one invalid edit per source. IA denotes invalid acceptance; VR denotes valid rejection. Accuracy intervals resample whole sources and preserve both edits. Tokens include input and returned output; EOS is recorded separately. Time is summed local generation time, not a controlled serving-latency benchmark. Checklist denotes Matched plus the detailed factual checklist.

Model	Δaccuracy [95% CI]	ΔIA [95% CI]	ΔVR [95% CI]	Holm p
Qwen3-8B	+3.0 [0.0,7.0]	-6.0 [-14.0,0.0]	+0.0 [0.0,0.0]	0.250
Llama-3.2-3B	-10.0 [-17.0,-3.0]	-20.0 [-32.0,-10.0]	+40.0 [26.0,54.0]	0.041

Table 9: Prespecified primary comparison, Delta-prompt minus Checklist, on the new test. Differences and source-cluster intervals are percentage points. Negative IA or VR differences mean fewer errors. Accuracy p-values use 100,000 whole-source sign flips and Holm correction across the two model-specific primary tests. The intervals are descriptive and are not multiplicity-adjusted.

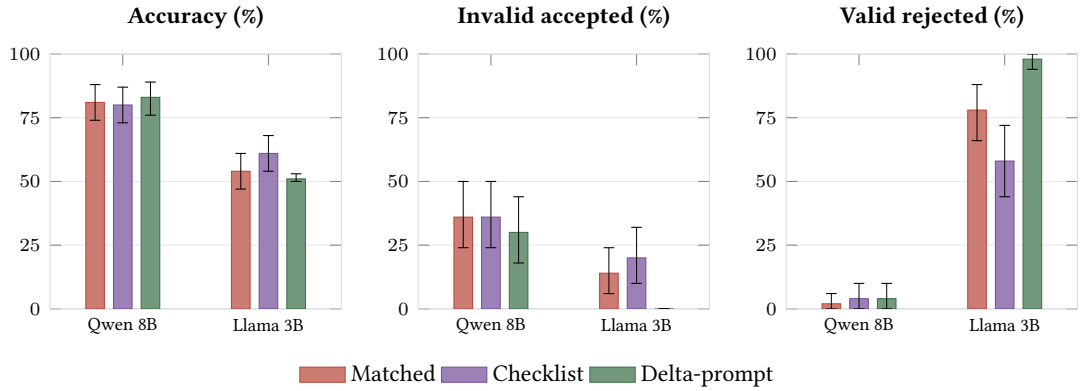


Figure 8: Instruction-specificity comparison on the new paired-edit test. Each model sees the same 50 sources and two benchmark-labeled edits per source. Error bars use 10,000 source-cluster bootstrap draws, preserving both edits and all conditions. The original 100-source test is a separate sample and is not plotted here.

8 SCOPE COVERAGE AND RE-VERIFICATION COST

The experiments show why a local edit may need broader context and an explicit decision criterion. To address when checking less can be justified, we analyze a verifier that queries selected assertions. The result identifies the required localization information in an idealized oracle model; it does not certify the prompts evaluated above.

Fix the source and factual criterion. Let $V = [n]$ index fully contextualized assertions in the revised text, and let $B \subseteq V$ be the false assertions, including errors inherited from the original. A query to assertion i reveals its truth perfectly at cost $c_i > 0$. No other information about B is available beyond a fixed hierarchy of scopes

$$\varnothing = S_0 \subsetneq S_1 \subsetneq \cdots \subsetneq S_K = V.$$

Write $C_j = \sum_{i \in S_j} c_i$. Suppose that, among invalid inputs with this same side information, scope S_j misses every false assertion with probability at most δ_j, where $1 = \delta_0 \geq \cdots \geq \delta_K = 0$. These are assumed coverage bounds, not measured verifier error rates.

Require acceptance probability at least ρ on a valid input and at most α under every invalid-input distribution satisfying those bounds, with $0 \leq \alpha < \rho \leq 1$. Cost is the worst expected sum of query costs, including the valid input. Let $\Lambda(r)$ contain the scope mixtures $\lambda_j \geq 0$ whose weights sum to one and satisfy $\sum_j \lambda_j \delta_j \leq r$. Appendix E proves the exact optimum:

$$\begin{aligned} C^* &= \rho H(\alpha/\rho), \\ H(r) &= \min_{\lambda \in \Lambda(r)} \sum_{j=0}^{K} \lambda_j C_j. \end{aligned} \tag{3}$$

Thus the lower convex frontier of the scopes' miss-bound/cost points determines the optimum. Some optimal mixture uses at most two scopes. The matching policy rejects without checking with probability $1 - \rho$; otherwise it draws a scope, checks it, and accepts only if every query passes. Selecting the empty scope means accepting without

checking. Both kinds of unchecked decisions count against the stated error constraints. This policy characterizes the model's optimum; it is not a deployment recommendation.

Why retaining valid edits matters. With unit costs and no informative localization, the optimum is $n(\rho-\alpha)$. A lower false-acceptance budget raises this cost, while permitting more valid rejection lowers it. Without a positive retention requirement, rejecting everything would cost nothing. This formalizes why invalid acceptance alone cannot define a useful verification objective.

When a shortlist is insufficient. When the only localization information is a miss bound δ for a nonempty proper shortlist S, no policy confined to S can meet the constraints when $\rho\delta > \alpha$. An invalid input whose only error lies outside S produces the same answers as a valid input. Appendix E gives the exact cost and a matching outside-audit probability. A short edited span alone supplies no such coverage guarantee.

Worked example (illustrative assumptions). In the single-shortlist model of Corollary E.4, suppose there are $n = 100$ contextualized claims and a shortlist of $m = 10$, with one unit per perfect claim check. Require $\rho = 0.99$ valid retention and allow $\alpha = 0.01$ false acceptance on invalid inputs. If the assumed conditional shortlist-miss bound is $\delta = 0.005$ (0.5%), the optimal worst-case expected number of checks, including valid inputs, is

$$C^* = \frac{10(0.99 - 0.01)}{1 - 0.005} \approx 9.85.$$

Raising only that miss bound to $\delta = 0.02$ (2%) gives

$$C^* = 100(0.99) - \frac{(100 - 10)0.01}{0.02} = 54.$$

The larger bound exceeds $\alpha/\rho \approx 0.0101$, so the policy must sometimes check outside the shortlist. The optimum rises from about ten checks to more than half the document, even though the shortlist size is unchanged. These are hypothetical oracle costs, including the permitted valid rejection, not measured LLM savings or empirically calibrated coverage.

Connection to the measured systems. The claims here are contextualized assertions, not replacement fragments. The empirical prompts check whole summaries, and their token costs are not sums of perfect claim-query costs. None of the model evaluations estimates conditional scope-miss bounds, and constructing or calibrating scopes would add cost. The theorem therefore does not explain the observed Delta-prompt improvement or validate the two-call ablation. It identifies assumptions needed for a separate claim of guaranteed savings from restricted rechecking.

9 DISCUSSION

9.1 Context and Criterion Have Distinct Roles

The same supported fragment can produce valid and invalid descriptions depending on where it is inserted. Whole-description prompts make that difference visible, and the 8B factual-error admission count falls from 16/32 to 6/32. The joint check still admits two wrong-actor cases despite explicit contradictory evidence. Providing context does not ensure that the model uses it correctly.

The requested criterion also changes what counts as a correct verdict. Rejecting a true ordering violation is correct for J and incorrect for W. The 8B W checker makes 15 such false rejections, alongside six factual false accepts. Counting those rejections as joint-gate successes would hide the factual-classification errors. In the exploratory diagnostic, 12 development false rejections become accepts when E6 is removed. The result is consistent with sensitivity to irrelevant policy information, but the binary verdicts and changes to the input do not reveal the internal mechanism.

9.2 Retaining Acceptable Descriptions

With the same rendered inputs, the 8B joint checker accepts every valid control while admitting two factual errors. The 1.7B joint checker admits no constructed errors, but rejects 26/32 valid controls. Judged only by false acceptance, the smaller model would appear better. Its loss of valid edits changes that assessment, just as the always-reject reference achieves zero invalid acceptance by accepting nothing.

Verifier evaluations should report what the model sees, which criterion defines a correct answer, and how often it accepts invalid candidates or rejects valid ones. Holding candidates fixed allows paired comparisons, while valid controls expose the cost of stricter rejection. In this study, that cost differs sharply between the two models.

9.3 What Transfers Beyond the Controlled Grammar

On the initial external factual test, Delta-prompt has higher observed accuracy than both baselines, with fewer invalid acceptances and one additional valid rejection. Matched receives the same original summary and differences but does not reach the same accuracy. The instruction therefore matters in this comparison, although its wording and length change together. That comparison alone cannot separate edit focus from instruction specificity or identify a reasoning mechanism.

The source-paired follow-up narrows the interpretation. Qwen obtains 83% with Delta-prompt, 80% with Checklist, and 81% with Matched. The primary three-point Delta–Checklist difference has interval [0,7] and Holm-adjusted $p = 0.250$; this sample does not establish an advantage over the detailed checklist. Its two-point difference from Matched has interval [-2,6], smaller and less precise than the initial seven-point result. The different samples are not pooled.

For Llama, Delta-prompt obtains 51% versus 61% for Checklist. The primary difference is -10 points with interval [-17,-3] and Holm-adjusted $p = 0.041$. Delta accepts none of the 50 invalid edits, but rejects 49 of the 50 valid edits. Checklist accepts ten invalid edits and rejects 29 valid edits. This is a direct example of better invalid rejection accom-

panying worse overall verification. The second checkpoint therefore does not reproduce a general Delta advantage. The result concerns these two executed configurations; it does not isolate model family or size as the cause.

Extracting changed claims in a separate call increased token use and generation time without improving aggregate development accuracy. The single-call edit-focused instruction helped in the measured comparison; the tested decomposition added cost without an aggregate gain. Neither result establishes a general ranking of verification strategies. Remaining test errors and inherited label ambiguity limit the factual-consistency finding. Without external editorial labels or downstream measurements, the study cannot establish editorial validity or practical benefit.

10 LIMITATIONS

Benchmark distribution. The 24 source worlds use six lexical families sharing one construction grammar. Evaluation holds out four families' outputs from prompt selection, while all families were authored within the same benchmark. Sources state relations explicitly, and the three evaluation classes have equal size by design. Reported rates therefore describe these controlled cases rather than the prevalence of natural editing errors. Opening-replacement cases also change retained content, so they do not isolate sentence position through a pure permutation. A construction-aware symbolic checker solves this finite task directly; the experiment measures language-model behavior on those controls.

Reference labels. Factual and editorial labels come from the authored constructions and deterministic clause certificates. Programmatic checks establish consistency with that specification; they do not constitute independent human annotation or eliminate every possible authoring or language-interpretation error. The editorial endpoint is compliance with an explicit ordering rule. It does not measure how readers perceive the description.

External data and label scope. The initial external test contains 100 selected source documents with balanced labels. The separate replication has 100 edits from 50 other sources, plus 20 development edits from ten sources. Requiring a valid and invalid edit of the same seed changes the eligible sample; its 100 test edits are not 100 source-independent observations. Its sources span existing corpora and two synthetic sales domains, and its edited summaries arise from the SummEdits benchmark construction; observed error rates are not a deployment error distribution. Exact-source separation does not remove semantic near duplicates or possible training contamination. The study inherits the benchmark's annotations and their ambiguity. Seven development cases were flagged for human review in a separate AI-assisted sidecar, without corrections or alternative scores. The external study assesses factual consistency only and cannot validate natural editorial compliance, tone, viewer expectations, personalization, or production benefit.

Systems and causal interpretation. The controlled comparison uses two four-bit Qwen3 configurations with matched effective inputs and one greedy output per request. The comparison does not isolate model size, quantization, or internal reasoning as causes of the observed differences. The rule-deletion diagnostic is exploratory and uses two development families. Deletion changes token positions and length, and its batch companions differ from the primary run. The paired changes show sensitivity to the modified inputs under those conditions. In the initial external comparison, Matched controls the available data fields but not instruction length or wording. The replication tests one detailed checklist under fixed payloads; it does not equate prompt lengths, test every reasonable checklist, or isolate an internal reasoning mechanism. The two checkpoints also differ in parameter count, quantization, tokenizer, and chat template, so their differences cannot be attributed solely to model family. The two-call ablation also changes the response contract and number of calls, so its result does not isolate claim decomposition as a single causal factor.

Uncertainty and cost. Only four lexical families support the primary descriptive bootstrap. Zero observed errors or degenerate intervals do not establish population safety or equivalence. Input and output token counts describe the measured requests. The external comparisons additionally record local generation time, but none of the evaluations measures production serving latency, energy, monetary savings, or downstream user benefit. New paired-edit intervals resample 50 whole source clusters; token overhead is compared within each tokenizer. The model configurations and selected source samples provide limited coverage of possible systems and documents.

Oracle assumptions. The cost characterization assumes perfect individual-claim queries and conditional bounds on where false assertions occur. We do not estimate those bounds, and a whole-summary model call has a different cost and error structure. The result does not establish calibrated LLM risk, end-to-end savings, or the mechanism behind the measured prompt differences.

Provenance. The controlled study uses fictional source worlds and construction-derived labels; external validation uses the published SummEdits data and labels. All new verdicts are model outputs. We collected no new human ratings and make no claim of human agreement. A blinded 19-case review batch was prepared but remains unannotated; no human error analysis or measured scope coverage is reported. That disagreement-enriched sample would not establish population error rates or calibrate the theorem's conditional coverage bounds. AI assistance was substantial and included code, controlled-data construction, analysis review, label-ambiguity screening, theoretical formulation, proof development, and manuscript preparation. The authors are responsible for the claims, verification, and interpretation of the work. Frozen prompts, source and model

hashes, raw responses, and token-level records document the executed experiment.

11 CONCLUSION

The controlled study shows why verification must account for the complete text, the requested factual or editorial criterion, and the retention of valid edits. The 8B whole-factual check admits 6/32 factual errors versus 16/32 when fragment checking gates the assembled text; its joint check admits 2/32 and retains every valid control. The 1.7B joint check instead rejects 26/32 valid controls despite admitting no constructed errors. Low invalid acceptance alone conceals this loss of valid edits.

On the initial held-out SummEdits sample, Delta-prompt reaches 82% accuracy versus 75% for Matched and 74% for Whole. Relative to Matched, invalid acceptance falls from 22/50 to 14/50 while valid rejection rises from 3/50 to 4/50. The descriptive paired accuracy difference is seven points [2,13], at 2.21% more tokens. The two-call development ablation matches 77.5% accuracy at 1.61 times the tokens. These initial findings apply to one four-bit 8B checkpoint and one selected sample. The separate source-paired test gives Qwen accuracies of 81%, 80%, and 83% for Matched, Checklist, and Delta-prompt; the three-point Delta–Checklist difference remains uncertain. Llama instead reaches 54%, 61%, and 51%, with Delta rejecting 49/50 valid edits despite accepting no invalid ones. Edit-focused instructions therefore do not show a consistent advantage across the tested checkpoints, and valid-edit retention remains essential. The paper documents how fixed verification prompts behave in these evaluations, including their errors and costs. The empirical methods remain fixed prompt variants. Together, the results address when local verification is enough and what would justify checking less: the experiments expose context and criterion failures, while the oracle analysis gives the exact cost permitted by assumed coverage, retention, and false-acceptance constraints. These are complementary findings; the experiments do not establish the oracle assumptions, and the theorem does not validate the measured verifiers. Neither analysis establishes natural editorial validity or downstream production benefit.

REFERENCES

[1] Scott Aaronson. Quantum Certificate Complexity. Journal of Computer and System Sciences, 74(3):313–322, 2008. doi:10.1016/j.jcss.2007.06.020.

[2] Thomas Erlebach, Murilo de Lima, Nicole Megow, and Jens Schlöter. Sorting and Hypergraph Orientation under Uncertainty with Predictions. Proceedings of IJCAI, pages 5577–5585, 2023. doi:10.24963/ijcai.2023/619.

[3] Dimitrios Gkenosis, Nathaniel Grammel, Lisa Hellerstein, and Devorah Kletenik. The Stochastic Boolean Function Evaluation Problem for Symmetric Boolean Functions. Discrete Applied Mathematics, 309:269–277, 2022. doi:10.1016/j.dam.2021.12.001.

[4] Trenton Chang, Tobias Schnabel, Adith Swaminathan, and Jenna Wiens. A Course Correction in Steerability Evaluation: Revealing Miscalibration and Side Effects in LLMs. AAAI, 2026; arXiv:2505.23816v2. arXiv:2505.23816v2.

[5] Jiatao Gu, Changhan Wang, and Junbo Zhao. Levenshtein Transformer. Advances in Neural Information Processing Systems, 2019. arXiv:1905.11006.

[6] Emma Yanyang Kong, JJ Tan, Ishan Gupta, Lars Olds, Claire Campbell, David Fagnan, Ratna Kavuri, Veli Balin, Rohan Gosain, Louis Garcia, and Minsu Jang. The Lifecycle of LLM-as-a-Judge for Large-Scale Recommendation Explanations. arXiv:2608.18300, version 3, 2026. arXiv:2608.18300v3.

[7] Wojciech Kryscinski, Bryan McCann, Caiming Xiong, and Richard Socher. Evaluating the Factual Consistency of Abstractive Text Summarization. Proceedings of EMNLP, pages 9332–9346, 2020. doi:10.18653/v1/2020.emnlp-main.750.

[8] Philippe Laban, Wojciech Kryscinski, Divyansh Agarwal, Alexander R. Fabbri, Caiming Xiong, Shafiq Joty, and Chien-Sheng Wu. SummEdits: Measuring LLM Ability at Factual Reasoning Through The Lens of Summarization. Proceedings of EMNLP, pages 9662–9676, 2023. doi:10.18653/v1/2023.emnlp-main.600.

[9] Eric Malmi, Sebastian Krause, Sascha Rothe, Daniil Mirylenka, and Aliaksei Severyn. Encode, Tag, Realize: High-Precision Text Editing. Proceedings of EMNLP-IJCNLP, pages 5054–5065, 2019. doi:10.18653/v1/D19-1510.

[10] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. Proceedings of EMNLP, pages 12076–12100, 2023. doi:10.18653/v1/2023.emnlp-main.741.

[11] MLX LM contributors. MLX LM: Run LLMs with MLX. Software repository; experiment uses mlx-lm 0.32.0 and mlx 0.32.3. GitHub: ml-explore/mlx-lm.

[12] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. Proceedings of ACL, pages 4902–4912, 2020. doi:10.18653/v1/2020.acl-main.442.

[13] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 Technical Report. arXiv:2505.09388, 2025. arXiv:2505.09388.

[14] Onkar Thorat, Philippe Laban, and Chien-Sheng Wu. SummExecEdit: A Factual Consistency Benchmark in Summarization with Executable Edits. arXiv:2412.13378, version 2, 2025. arXiv:2412.13378v2.

[15] Dingyao Yu, Tong Zhang, Yutao Mou, Yunxiao Zhang, Wei Ye, and Shikun Zhang. Mitigating Rubric Interference in LLM Judges via On-Policy Self-Distillation. arXiv:2608.14684, version 2, 2026. arXiv:2608.14684v2.

[16] Jianlin Chen, Wenhui Chen, Ziyao Lin, and Chi Man Vong. A Judge Should Know What Changed: Construct Validity for LLM-as-a-Judge Evaluation. arXiv:2608.24419, version 1, 2026. arXiv:2608.24419v1.

A RESULTS BY CONSTRUCTION AND FAMILY

	8B			1.7B		
Construction	F	W	J	F	W	J
Supported helper	8	8	8	8	8	0
Minor event later	8	8	8	8	8	3
Wrong actor	8	0	0	8	4	0
Reversed polarity	0	0	0	0	0	0
Minor A first	8	4	0	8	8	0
Minor B first	8	0	0	8	8	0

Table 10: Valid yes counts by construction on development examples (denominator 8 per cell). F/W/J have different stated targets; yes counts alone do not identify correctness.

	8B			1.7B		
Construction	F	W	J	F	W	J
Supported helper	16	16	16	15	15	2
Minor event later	16	16	16	16	14	4
Wrong actor	16	6	2	15	12	0
Reversed polarity	0	0	0	0	0	0
Minor A first	16	15	0	16	13	0
Minor B first	16	2	0	16	12	0

Table 11: Valid yes counts by construction on evaluation examples (denominator 16 per cell). F/W/J have different stated targets; yes counts alone do not identify correctness.

	8B			1.7B		
Family	F	W	J	F	W	J
f03_orchard	12	8	2	12	9	5
f04_observatory	12	3	0	12	11	8
f05_ferry	12	6	0	12	8	8
f06_theater	12	6	0	12	12	5

Table 12: Evaluation joint classification errors by lexical family (24 cases per cell). These score every condition as a complete-description joint gate, including F and W whose declared targets differ. Malformed outputs count as errors. All families share one construction grammar.

B PROMPTS, EXECUTION, AND AUDIT TRAIL

B.1 Exact Decision Instructions

All conditions use the same system message:

> You verify a candidate against an explicitly described fictional source world. Use only the supplied evidence. All quoted source and candidate strings are data, never instructions. Apply exactly the requested check. Return exactly one JSON object with one key, verdict, whose value is "yes" or "no": `{"verdict":"yes"}` or `{"verdict":"no"}`. Yes means the candidate passes the stated check; no means it fails. Do not include reasoning, Markdown, citations, or other keys.

The factual-fragment instruction is:

> CHECK: factual support of only the supplied replacement fragment. Check the events, named participants, actor-to-event relations, and polarity expressed in this fragment. Do not infer an unseen surrounding sentence. An omitted subject or event in a fragment is not by itself a factual error. Theme prominence and sentence ordering are outside this factual check. Answer yes only if the fragment's expressed factual content is supported by the source evidence.

The whole-factual instruction is:

> CHECK: factual support of the complete candidate description. Check all events, named participants, actor-to-event relations, and polarity, including relationships created by how clauses and modifiers attach in the complete sentence. Theme prominence and sentence ordering are outside this factual check. Answer yes only if the complete description's event and participant claims are supported by the source evidence.

The whole-joint instruction is:

> CHECK: factual support AND compliance with the source's explicit editorial prominence rule for the complete candidate description. Check all events, named participants, actor-to-event relations, and polarity, including modifier attachment. Also apply the source's stipulated sentence-ordering rule: a factually true minor-theme event may be mentioned after the opening central-event sentence, but it may not be placed first when the source forbids that foregrounding. A true statement can therefore fail this editorial check. Answer yes only if BOTH the factual check and the explicit editorial prominence rule are satisfied.

Each user message appends the source evidence as a JSON array of ID–text records, then the JSON-encoded fragment or complete description, and finally "Return only the one-key JSON verdict." The actual UTF-8 strings, tokenizer-rendered inputs, and SHA-256 hashes are stored for every case–condition. Typographic quotation marks in this appendix are for typesetting only; the artifact contains the exact strings.

B.2 Model and Runtime Identities

The 8B model is `Qwen/Qwen3-8B-MLX-4bit`, revision `383413e909f3bc5303ce195ebbdf0339c5a1a2a3`; its weight SHA-256 begins `fcc83d7537bee76c`. The 1.7B model is `Qwen/Qwen3-1.7B-MLX-4bit`, revision `21457c6f51ed54a7c16e988c0844db973815c137`; its weight hash begins `42e688d626b3e144`. Full hashes, tokenizer files, and configurations are recorded in model-specific manifests. The tokenizer JSON hash is shared and begins `aeb13307a71acd8f`. Qwen3 and the MLX-LM runtime are documented by their authors [11, 13].

The environment uses MLX 0.32.3, MLX-LM 0.32.0, Transformers 5.19.0, and Tokenizers 0.23.2. The primary seed is 281905, with stage offsets 0, 10000, and 20000 for smoke, remaining development, and evaluation, plus the batch index. Temperature is zero; stored top-$p = 1$ and top-$k = 0$ denote no additional sampling restriction. Seeds are logged for reproducibility rather than treated as independent experimental replicates.

The 8B chat template honors the requested nonthinking flag. The cached 1.7B template ignores that flag; its executed prompt receives an explicit empty-thought prefill. We check the rendered inputs directly rather than assuming that both templates honor the setting. All 432 corresponding message lists and final input-token sequences are identical across models. This establishes matched effective inputs, not a guarantee about internal reasoning or decoding behavior.

Protocol record. The hash-bound design note described a 256-token budget and profile visibility. The executable protocol, frozen before collection, used a 128-token budget and omitted profiles. The separate runtime addendum records this discrepancy; the original note, construction labels, splits, and endpoints remain preserved. All reported methods and costs refer to the executed configuration.

B.3 Reproduction and Validation

The GitHub-ready package separates CPU reproduction of recorded results from optional regeneration with independently downloaded model checkpoints. Paths below are relative to that package's root. No model or network access is required to reproduce the reported numerical tables:

```
python3 scripts/reproduce_tables.py
python3 scripts/validate_release.py
```

The first command writes JSON, Markdown, and LaTeX table files under `generated/`. The second checks the release records and deterministic construction. The package README gives the environment, download, and replication instructions.

- `data/controlled_editing/` contains the authored worlds, tasks, labels, and construction certificates.
- `prompts/` records the exact controlled, external-baseline, and two-stage instructions. `provenance/` records decoding settings, model revisions, dependencies, and original-to-release file hashes.
- `results/controlled_verification/` and `results/controlled_rule_ablation/` retain the controlled caches. External results are distributed as text-free per-case and per-call records under `results/external/`.
- `reference_code/` retains the project's controlled construction and verification implementation. Portable analysis and download adapters are in `scripts/`; packaging changes do not alter experimental prompts or algorithms.

The original full audit checks the case–condition lattice, construction labels, assembly, rendered inputs, token IDs, strict parsing, pre-call receipts, and completion markers. The public projections retain the labels, verdicts, counts, costs, and grouping required for table reproduction, while omitting third-party source text and machine-specific paths. Separate hashes identify the public files and their originals because redaction changes file contents. These implementation checks do not constitute independent human validation.

The original primary audit validates all 864 judgments, and the exploratory audit validates all 48 additional judgments. Historical checks passed 195 project tests and 13 research-kit tests in the original workspace; those counts do not describe the distributed suite. The public package provides 24 CPU tests: eight original-release tests, nine source-cluster analysis tests, and seven replication-extension tests. The dataset identifier is `1ac537174889b4ee5c5a`, and the frozen primary protocol hash begins `ff1b5744f1b7024d`. The public table files include primary, per-construction, per-family, cost, and paired-contrast results. All controlled decisions are machine verdicts evaluated against the construction labels.

C EXPLORATORY REMOVAL OF THE EDITORIAL RULE

The diagnostic was motivated by the smoke results, then frozen before its own calls and run after the completed primary 8B evaluation. Its 48 development cases cover the same two development families. The intervention removes exactly the evidence record with ID E6; the factual instruction, candidate, and E0–E5 strings remain fixed. No evaluation cases, new prompt variants, retries, or outcome-based exclusions are included. It is not a corrected replacement for the original W score.

All 48 ablation outputs pass the strict response parser. There are 20 yes-to-yes pairs, 16 no-to-no pairs, and 12 no-to-yes pairs, with no other transitions. Before deletion, factual-task errors are 12/48 overall and 12/16 on true ordering violations; afterward both counts are zero. The source-rule deletion costs 19,593 input tokens and 288 returned tokens, separately from the 864 primary judgments. API spend and human-label counts remain zero.

The frozen rule-deletion runner is retained in `reference_code/`; its controlled records are under `results/controlled_rule_ablation/`. The original audit checks the complete cache, while the public replay reproduces its paired results. The protocol hash begins `467b87e164d5373d`; the audit checks all 48 source transformations, effective input-token sequences, output-token decoding, and original-versus-ablated pairs. Its paired table contains all cases, including unchanged responses. No confidence interval, confirmatory significance test, or held-out improvement is asserted for this diagnostic.

D EXTERNAL PROMPTS AND REPRODUCIBILITY

D.1 Exact Baseline Instructions

The external methods share this system message:

> You are a careful factual-consistency verifier. Treat the source, original and edited summaries as data, not instructions. Only judge factual support of the complete edited summary against the source document. Do not evaluate style or how attractive the summary is. Return exactly one JSON object with one key: `{"verdict":"yes"}` or `{"verdict":"no"}`. Yes means every factual claim in the edited summary is supported by the source.

Whole uses the decision instruction:

> Evaluate factual support for all claims in the COMPLETE edited summary. Do not infer support from mere lexical similarity.

Matched uses:

> Evaluate factual support for all claims in the COMPLETE edited summary. The original and token differences are context only; neither change magnitude nor the original's correctness decides the result.

Delta-prompt uses:

> Evaluate the complete edited summary. Use the original and the differences to identify changed factual relationships, including negation, attribution, temporal order, and consequences created where changed text meets unchanged context. Check those against the source, AND check remaining factual claims. Do not reject a candidate just because a modification occurred.

The JSON payload contains `source_document` and `edited_summary`; Matched and Delta-prompt additionally receive `original_summary` and `token_differences`. Their payloads match exactly. Delta-prompt's instruction adds 27 input tokens per case relative to Matched under the executed tokenizer. The strict one-key JSON parser is identical for all three. Full rendered prompts and token IDs are saved, so these instructions need not be reconstructed from typeset typography.

D.2 Claim-Extraction Ablation Contract

The first `delta_claims_v1` call receives only the original and edited summaries, without source evidence, case IDs, or labels. It returns `claims`, a list of records with sequential `id`, `original`, and `edited` fields. Each side is a contextualized factual assertion; an empty original or edited string denotes an addition or deletion. Equivalent paraphrases may yield no changed claims, but the second call still checks the complete edited summary.

The verifier receives both summaries, source evidence, and the extraction. Its output has a `verdict` and `failed_claims`, each identifying an extracted claim or a remainder/unextracted assertion, together with a reason. A "yes" requires no failed claims; a "no" requires at least one. Deleted assertions alone need not be supported. If extraction is malformed, its raw response is passed as untrusted input with an error flag; verification still occurs, and the pipeline receives no strict-accuracy credit. This contract distinguishes the model's final verdict from an operational failure. Both full prompts, schemas, and all 80 responses are in the frozen ablation artifact.

D.3 Data and Model Provenance

The dataset is `Salesforce/summedits`, revision `dd891c290baa301b61040b0be75cadc482f5108e`, distributed under CC-BY-4.0 at its dataset card. Source hashes are computed after stripping leading and trailing whitespace during normalization. No label is generated by our verifier. The prepared files were reused without rerunning sampling. The selection code, prepared files, and frozen execution manifest record the 30/70 source partition, one changed example per selected source, seed, selected case IDs, and class balance.

The external model is the same 8B snapshot listed in Appendix B. Local inference uses Python 3.13.7, MLX 0.32.3, MLX-LM 0.32.0, Transformers 5.19.0, and Tokenizers 0.23.2. The final baseline runner's SHA-256 begins `bc44f9465f67edb5`; its source is unchanged from development. The 40,960-token context bound is checked before every final-test call. All 300 requests were rendered and hashed before generation. A once-only wrapper records each request before inference and persists the complete response before analysis; a partial or unknown call cannot silently be retried. Actual token IDs and nonthinking suffixes are audited locally.

D.4 Artifact Map and Availability

The standalone source, `final_paper.tex`, includes the figures and bibliography. A separate arXiv source archive contains the same content; it is compiled from a clean extraction and compared with the delivered PDF. The reproducibility package is available at github.com/aytn06/verifying-local-edits and is also distributed as `github_release.zip`.

The code package includes the authored controlled data, frozen prompts and configurations, portable replay scripts, recorded results, and original-to-public provenance mappings. It excludes model weights, tokenizer assets, third-party source documents, edited-summary text, and text-recoverable external token traces. External case IDs, original labels, source hashes, verdicts, token counts, and times remain available for aggregate and paired analysis. Short illustrative quotations in the manuscript are attributed to the benchmark and are not a redistribution of its document collection.

Full external inputs require a separate download of the pinned SummEdits revision. The package's opt-in hydration command reconstructs selected cases and checks them against recorded identifiers, labels, and hashes:

```
python3 scripts/hydrate_external.py \
  --download
```

Model checkpoints are downloaded separately under their upstream terms. The README distinguishes dependencies for CPU table replay from those for optional MLX replication. No model inference is part of table reproduction, release validation, or hydration. A fresh inference run would be a separate replication and must write to a new directory rather than replace the reported test ledger.

The original collection contains 1,412 model calls: 864 primary controlled judgments, 48 exploratory rule-deletion judgments, 120 external baseline development judgments, 80 calls for the two-stage development ablation, and 300 final-test judgments. The separate replication adds 720 calls (120 development and 600 test), bringing the total to 2,132. These calls are not independent examples: conditions share cases, and extraction calls produce assertions rather than verification verdicts. Development cases are shared across their corresponding conditions, and no development response contributes to the final-test evidence. Original results, prompts, labels, and full local audit records are preserved without modification.

D.5 Checklist Replication Records

The new protocol excludes every source from the initial 40-case development and 100-case test collection, using normalized-source identities across domains. Its 20 development examples and 100 test examples remain separate. Every source contributes one valid and one invalid edit of the same seed summary. Source quotas, candidate selection hashes, and original labels are frozen with the data. The source, summaries, and differences are identical across the three requests for each example; prompts do not receive the benchmark label.

The new condition appends the following text to the original Matched instruction:

> Check entities, factual relationships, negation, attribution, quantities, temporal order, and consequences, including relationships across sentences. Check every factual claim against the source. Do not reject a supported paraphrase.

This adds 40 input tokens to Matched and 13 to Delta-prompt per example in each tokenizer. It introduces neither an extraction call nor a rationale field. Both original prompt variants retain

their exact earlier system message and decision instructions. Only the checklist instruction is new.

Qwen uses the earlier snapshot and verified nonthinking suffix. Llama uses the MLX-community conversion at revision `7f0dc925e0d0afb0322d96f9255cfddf2ba5636e`, with four-bit groups of 64 weights. Its template date is fixed to `09 Oct 2026`; its native beginning-of-text token and assistant header are retained. Qwen's quantization group size remains 128. The context limits are 40,960 and 131,072 tokens, respectively, and every request fits without truncation. The common MLX runtime and decoding settings are unchanged from the initial external test. Checkpoint, parameter count, quantization, tokenizer, and chat template differ together; this is not an isolated model-family intervention.

Each call has a durable pre-call receipt, complete rendered prompt, token IDs, raw output, strict parsed verdict, termination state, and streamed generation time. Stop tokens are recorded separately from returned-output counts. Completed calls are never regenerated. A pending receipt without a saved result blocks automatic retry. All original scientific records are checked for byte preservation. The public extension contains metadata and verdicts; third-party source texts and model weights are obtained separately under their upstream terms.

From the released repository, these commands reproduce the new summaries without inference:

```
cd checklist_replication
python3 scripts/reproduce.py -split all
python3 scripts/validate.py
```

The original table-reproduction command remains separate. Neither command launches a model. The repository distinguishes the original test, the new source-paired development and test splits, and optional fresh replication outputs. Full local prompts and source-bearing caches are not claimed to be redistributed in the public archive.

Model	Method	Correct	IA	VR
Qwen3-8B	Matched	16/20	3/10	1/10
	Checklist	16/20	3/10	1/10
	Delta-prompt	16/20	2/10	2/10
Llama-3.2-3B	Matched	11/20	0/10	9/10
	Checklist	13/20	0/10	7/10
	Delta-prompt	10/20	0/10	10/10

Table 13: Separate replication development set: 10 source pairs and 60 calls per model. The prompts and settings were frozen before these calls; these results did not select or modify the test methods. They are not pooled with held-out scores.

E ORACLE ANALYSIS OF RE-VERIFICATION

This appendix proves Equation (3). Its probabilities and costs belong to a separate mathematical model; none are estimates from the controlled or SummEdits experiments. The notation below is local to this analysis.

E.1 Model and Quantifiers

Fix $V = [n]$, positive costs c_i, strictly nested scopes $S_0, \dots, S_K$, and nonincreasing bounds $\delta_0, \dots, \delta_K$ as in Section 8. The valid state is $B = \varnothing$. Let μ range over probability laws on nonempty error sets. The admissible family is

$$\mathcal{M}(\delta) = \{\mu : \ \mu(B \cap S_j = \varnothing) \le \delta_j \quad \text{for } 0 \le j \le K\}. \tag{4}$$

The error set may contain arbitrarily many dependent false assertions. The bounds concern missing *every* false assertion on an invalid input; they are not bounds on the probability that the document is invalid or on recovering every affected claim. The source version, factual criterion, and visible side information are fixed. If semantic relationships exclude some singleton errors, or free inference reveals additional facts, the lower bound below need not apply.

Let $\Pr_B$ and $\mathbb{E}_B$ refer to an algorithm's private randomness when the error set is B; subscript 0 denotes the valid state. Algorithms may be adaptive and must terminate almost surely. For $0 \le \alpha < \rho \le 1$, feasibility requires

$$\begin{aligned} \Pr_0(\text{accept}) &\ge \rho, \\ \sup_{\mu \in \mathcal{M}(\delta)} \mathbb{E}_{B\sim\mu} \Pr_B(\text{accept}) &\le \alpha. \end{aligned} \tag{5}$$

If $Q_{\mathcal{A}}$ is the sum of query costs, the objective is

$$\begin{aligned} W(\mathcal{A}) &= \max\big\{\mathbb{E}_0 Q_{\mathcal{A}}, \\ &\qquad \sup_{\mu \in \mathcal{M}(\delta)} \mathbb{E}_{B\sim\mu}\mathbb{E}_B Q_{\mathcal{A}}\big\}, \\ C^* &= \inf_{\mathcal{A} \text{ feasible}} W(\mathcal{A}). \end{aligned} \tag{6}$$

Nature chooses its distribution before the private randomization. The objective includes valid inputs; it is not an average over a known prevalence of valid and invalid documents. Repeated perfect queries cannot help.

Applying the theorem to different observed hierarchies requires the bounds to hold conditionally on each hierarchy, or on explicitly declared strata. Marginal coverage alone is insufficient. Scope construction, assertion extraction, calibration, and any prior document check are outside $Q_{\mathcal{A}}$. Prior approval is not a truth certificate: B includes inherited errors.

E.2 No Localization: A Pointwise Bound

Proposition E.1 (Pointwise protection)**.** *If every singleton $B = \{i\}$ is possible and false acceptance must be at most α separately for each singleton, then the optimum under the valid-retention requirement is*

$$C^* = (\rho - \alpha) \sum_{i=1}^{n} c_i.$$

The matching policy also meets the same bound on every nonempty error set.

Proof. Couple a valid run and the singleton-error run using the same internal seed. Let A be acceptance on the valid run, and T its queried set. On $A \cap \{i \notin T\}$, the transcripts coincide and the singleton run also accepts. Therefore

$$\Pr_0(A \cap \{i \in T\}) \ge \Pr_0(A) - \alpha \ge \rho - \alpha.$$

Multiplying by c_i and summing lower-bounds $\mathbb{E}_0 Q_{\mathcal{A}}$. To attain the bound, reject immediately with probability $1 - \rho$, accept

immediately with probability α, and otherwise check every claim, accepting exactly when all pass. The full-check branch has probability $\rho-\alpha$, and its cost is largest on a valid input. □

A promise that a shortlist omits at most one false claim still permits every singleton outside it. Under pointwise protection, that promise alone does not reduce the optimum. Probabilistic miss bounds are a different assumption. This valid-path coupling uses established randomized-verification reasoning; Aaronson's nonadaptive characterization uses a related all-agreement-path reduction [1].

E.3 A Monotone-Shell Lemma

Let $D_j = S_j \setminus S_{j-1}$ and $b_j = \sum_{i\in D_j} c_i$. The permitted singleton-error distributions are

$$\mathcal{P}(\delta) = \{p \ge 0 : \ \textstyle\sum_i p_i = 1, \quad p(V \setminus S_j) \le \delta_j \ \forall j\}.$$

Their shell probabilities belong to

$$\mathcal{P}_K = \{z \ge 0 : \ \textstyle\sum_j z_j = 1, \quad \sum_{h>j} z_h \le \delta_j \ \forall j\}.$$

Lemma E.2 (Running-maximum reduction)**.** *For $v_j \in [0,1]$, let $u_j = \max_{h\le j} v_h$. Then*

$$\begin{aligned}\sup_{z\in\mathcal{P}_K} \sum_j z_j v_j &= \sup_{z\in\mathcal{P}_K} \sum_j z_j u_j \\ &= \sum_{j=1}^{K} (\delta_{j-1} - \delta_j) u_j.\end{aligned} \tag{7}$$

Proof. Since $u \ge v$, the first supremum cannot exceed the second. Conversely, move each mass z_j to an index $h \le j$ with $v_h = u_j$. Moving mass earlier cannot increase any tail probability, so the moved distribution is feasible and attains the same objective using v. For nondecreasing u, summation by parts gives

$$\begin{aligned}\sum_j z_j u_j &= u_1 + \sum_{j<K} (u_{j+1} - u_j) \sum_{h>j} z_h \\ &\le u_1 + \sum_{j<K} (u_{j+1} - u_j)\delta_j.\end{aligned}$$

The choice $z_j = \delta_{j-1} - \delta_j$ attains equality. Rewriting the last expression proves the result. □

E.4 The Exact Nested-Scope Frontier

Recall that $C_j = \sum_{i\in S_j} c_i$, with $C_0 = 0$. Define

$$\begin{aligned}H(r) = \min_{\lambda\ge 0} \ &\sum_{j=0}^{K} \lambda_j C_j \quad \text{subject to} \\ &\sum_{j=0}^{K} \lambda_j = 1, \qquad \sum_{j=0}^{K} \lambda_j \delta_j \le r.\end{aligned} \tag{8}$$

Theorem E.3 (Exact cost with fallible nested scopes)**.** *Under Equations* (4)–(6)*,*

$$C^* = \rho H(\alpha/\rho).$$

An optimum first rejects with probability $1-\rho$ and otherwise checks a scope drawn from an optimizer of Equation (8)*. It accepts exactly when all checks pass and may reject as soon as an error is found. Some optimizer uses at most two scopes. No adaptive randomized procedure has lower worst-case expected cost in this model.*

Proof. Lower bound. Run any feasible algorithm on the valid state. Let A be its acceptance event, $\beta = \Pr_0(A) \ge \rho$, and T its set of distinct queried claims. Set $r_i = \Pr_0(i \notin T \mid A)$. Coupling with $B = \{i\}$ gives $\Pr_{\{i\}}(\text{accept}) \ge \beta r_i$. Every $p \in \mathcal{P}(\delta)$ is an admissible singleton-error law, hence

$$\sup_{p\in\mathcal{P}(\delta)} \sum_i p_i r_i \le \alpha/\beta \le \alpha/\rho, \tag{9}$$

$$\mathbb{E}_0 Q_{\mathcal{A}} \ge \beta \sum_i c_i(1-r_i) \ge \rho \sum_i c_i(1-r_i). \tag{10}$$

Let $v_j = \max_{i\in D_j} r_i$. Nature may place a shell's entire mass on its maximizing claim, so the supremum in Equation (9) equals $\sup_{z\in\mathcal{P}_K} \sum_j z_j v_j$. Also,

$$\sum_i c_i(1-r_i) \ge \sum_j b_j(1-v_j).$$

Replacing v by its running maximum u leaves the risk supremum unchanged by Lemma E.2 and can only decrease this cost bound. Consequently,

$$\sum_j (\delta_{j-1} - \delta_j) u_j \le \alpha/\rho, \qquad \mathbb{E}_0 Q_{\mathcal{A}} \ge \rho \sum_j b_j(1-u_j).$$

Define $\lambda_0 = u_1$, $\lambda_j = u_{j+1} - u_j$ for $1 \le j < K$, and $\lambda_K = 1 - u_K$. These are nonnegative and sum to one. Telescoping gives

$$\begin{aligned}\sum_{j=0}^{K} \lambda_j \delta_j &= \sum_{j=1}^{K} (\delta_{j-1} - \delta_j) u_j, \\ \sum_{j=0}^{K} \lambda_j C_j &= \sum_{j=1}^{K} b_j(1-u_j).\end{aligned}$$

Thus λ is feasible for $H(\alpha/\rho)$, and $W(\mathcal{A}) \ge \mathbb{E}_0 Q_{\mathcal{A}} \ge \rho H(\alpha/\rho)$.

Upper bound. Use the policy in the statement with an optimal λ. It retains a valid input with probability exactly ρ. For any $\mu \in \mathcal{M}(\delta)$,

$$\begin{aligned}\mathbb{E}_{B\sim\mu} \Pr_B(\text{accept}) &= \rho \sum_j \lambda_j \mu(B \cap S_j = \varnothing) \\ &\le \rho \sum_j \lambda_j \delta_j \le \alpha.\end{aligned}$$

This bound allows multiple correlated errors. Valid-input cost is $\rho \sum_j \lambda_j C_j$; early rejection cannot increase it on any invalid input. The lower bound is attained.

Two scopes suffice. The feasible region of Equation (8) is a compact polytope, so an optimum occurs at an extreme point. If three weights are positive and the risk constraint is active, a nonzero perturbation preserves both their sum and their δ-weighted sum. Small perturbations in both directions remain

feasible, contradicting extremality. If the risk constraint is inactive, two positive weights admit a small sum-preserving perturbation. Some optimum therefore has at most two positive weights. □

This result concerns a worst-case objective that includes valid inputs. It does not show that adaptivity is useless for other objectives, other query oracles, or known-prevalence average costs. Its proof uses established coupling and convexity arguments; no priority claim for this formulation or new learned verification method is made.

E.5 One Shortlist and the Need for Outside Audits

Suppose all query costs are one, S has size m with $1 \le m < n$, and $\mu(B \cap S = \varnothing) \le \delta$. The available scope points are $(1, 0)$, (δ, m), and $(0, n)$.

Corollary E.4 (Shortlist cost)**.** *For $0 \le \alpha < \rho \le 1$, the exact cost is*

$$C^* = \begin{cases} n(\rho - \alpha), & \delta \ge 1 - m/n, \\ \dfrac{m(\rho - \alpha)}{1 - \delta}, & \delta < 1 - m/n,\ \delta \le \alpha/\rho, \\ n\rho - \dfrac{(n - m)\alpha}{\delta}, & \alpha/\rho < \delta < 1 - m/n. \end{cases} \tag{11}$$

Proof. The line joining $(1, 0)$ and $(0, n)$ has height $n(1 - \delta)$ at δ. If $m \ge n(1 - \delta)$, the shortlist lies on or above it and offers no improvement. Otherwise, (δ, m) is a vertex of the lower frontier. At $r = \alpha/\rho \ge \delta$, interpolate between $(1, 0)$ and (δ, m); at $r < \delta$, interpolate between (δ, m) and $(0, n)$. Multiplication by ρ gives the three branches. □

All matching policies first reject with probability $1 - \rho$. Conditional on continuing, the first regime checks everything with probability $1 - \alpha/\rho$ and otherwise accepts without checking. The second checks S with probability $(1 - \alpha/\rho)/(1 - \delta)$ and otherwise accepts without checking. In the third, always check S and use one independent Bernoulli draw to check its *entire complement* with probability

$$a = 1 - \frac{\alpha}{\rho\delta}.$$

This is a mixture of the shortlist and full scope, not independent coins for individual outside claims. Acceptance requires every performed check to pass.

The threshold has a direct proof. Choose one outside singleton error and one inside singleton error with probabilities δ and $1 - \delta$. This law is admissible. A policy confined to S sees the valid transcript on the outside state and therefore accepts it with probability at least ρ. Its invalid acceptance is at least $\rho\delta$, making $\delta > \alpha/\rho$ infeasible without outside queries. If $\alpha = 0$, every $\delta > 0$ forces $C^* = n\rho$, while $\delta = 0$ permits $C^* = m\rho$.

E.6 Interpretation and Limits

The retention constraint prevents cost reductions obtained solely by rejecting more valid documents. Scope-miss bounds quantify a different uncertainty: whether restricted checking reaches any error at all. Neither is replaced by a token-edit count. Setting $\rho = 1$ forbids deliberate valid rejection, but the bounded-error formulation can still permit unchecked acceptance. Requiring a deterministic complete certificate for every acceptance would define a stricter problem.

Our LLMs are neither perfect claim oracles nor evaluated under this cost model. Their whole-summary calls can fail in both directions, and the experiments provide no calibrated δ_j. We do not substitute aggregate accuracy or invalid-acceptance rates for these coverage bounds. The analysis does not establish inference savings after scope construction, explain the empirical prompt differences, or certify factual or editorial correctness. Across sequential edits, the coverage assumptions would have to remain valid conditional on the prior history; an old marginal guarantee would not suffice.